Practical – 4

JavaScript DOM Manipulation, Event Handling, and UI Interactivity

Problem Definition

Add dynamic UI components such as a collapsible FAQ, modal popup, image/content slider, notification banner, hamburger menu, and light/dark theme switcher to the StudentHub portal using JavaScript. Use static FAQ and event content as test data.

A. Theory

1. How are DOM elements selected and modified?

Answer: Elements are selected with document.querySelector(), document.querySelectorAll(), or document.getElementById(). They are modified through properties and methods such as textContent, classList.add/remove/toggle, setAttribute, dataset, hidden, and style. This practical mostly toggles CSS classes (for example .open and .hide) so that all visual rules stay in the stylesheet.

2. Are event listeners attached correctly?

Answer: Listeners are attached with addEventListener() after the DOM is ready (the script uses defer and a DOMContentLoaded handler). The FAQ uses event delegation: one click listener on the list handles every question. Keyboard events (Escape, Tab, arrow keys) are handled where needed. Correct attachment is verified in the browser console and in the Event Listeners panel of the developer tools.

3. Is localStorage used for remembering theme choice?

Answer: Yes. When the theme button is clicked, the new theme is saved with localStorage.setItem(). On page load it is read with getItem(); if nothing is saved, the operating system preference (prefers-color-scheme) is used. Reads and writes are wrapped in try/catch so the page still works if storage is blocked.

4. Does interactivity improve usability without breaking accessibility?

Answer: Yes, when native button elements are used and ARIA states (aria-expanded, aria-controls, aria-pressed) are kept in sync. The modal moves focus inside, traps Tab, closes with Escape, and returns focus to the trigger. The slider can be paused, and transitions are disabled for users who prefer reduced motion. Focus outlines stay visible and both themes keep readable contrast.

5. What is the DOM?

Answer: The Document Object Model is the browser’s tree representation of an HTML page. JavaScript can read and change this tree to update content, structure, and styles without reloading the page.

6. What is the difference between querySelector and querySelectorAll?

Answer: querySelector returns the first matching element (or null). querySelectorAll returns a static NodeList of all matches, which can be converted to an array with the spread operator and looped over.

7. What is event bubbling and event delegation?

Answer: Bubbling means an event travels from the clicked element up through its parents. Delegation uses this: one listener on a parent checks event.target (with closest()) and handles events for many children, including elements added later.

8. Why is classList preferred over changing style directly?

Answer: Classes keep presentation in CSS, are easy to toggle, and allow transitions. Inline styles scatter design rules in JavaScript and are harder to maintain and override.

9. What is the difference between localStorage and sessionStorage?

Answer: localStorage keeps data until it is cleared by the user or the script. sessionStorage keeps data only for the current tab session. Both store strings only, so objects must be saved with JSON.stringify() and read with JSON.parse().

10. Why are ARIA attributes such as aria-expanded used?

Answer: They expose the state of custom widgets to assistive technology. For example, aria-expanded tells a screen reader whether an FAQ answer is open or closed, and aria-modal tells it that content behind a dialog is inactive.

B. Implementation / Output / Result

Extend the StudentHub project from Practical 3. Add a js folder, update the shared stylesheet, and add the interactive components to index.html. The same script can be linked on every page because each function checks that its elements exist before running.

1. Project folder structure

```
StudentHub/
├── index.html      (updated with all components)
├── about.html
├── register.html
├── dashboard.html
├── events.html
├── contact.html
├── feedback.html
├── css/
│   └── style.css   (updated with theme variables and component styles)
└── js/
    └── script.js   (new)
```

2. Home page with components (save as index.html)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>StudentHub | Interactive Home</title>
  <link rel="stylesheet" href="css/style.css">
  <script src="js/script.js" defer></script>
</head>
<body>
  <a class="skip-link" href="#main-content">Skip to main content</a>

  <!-- Notification banner -->
  <div class="notice-banner" id="notice-banner" role="status">
    <p>Registration for TechFest 2026 is open until 30 November.</p>
    <button class="banner-close" id="banner-close" type="button" aria-label="Dismiss
notification">&times;</button>
  </div>

  <header class="site-header">
    <div class="container header-row">
      <p class="brand"><strong>StudentHub</strong></p>
      <div class="header-controls">
        <button class="icon-btn" id="theme-toggle" type="button" aria-label="Dark theme" aria-
pressed="false">Dark</button>
        <button class="icon-btn menu-toggle" id="menu-toggle" type="button" aria-label="Toggle
navigation menu" aria-expanded="false" aria-controls="main-nav">&#9776;</button>
      </div>
      <nav class="main-nav" id="main-nav" aria-label="Main navigation">
        <ul class="nav-list">
          <li><a href="index.html" aria-current="page">Home</a></li>
          <li><a href="about.html">About</a></li>
          <li><a href="register.html">Register</a></li>
          <li><a href="dashboard.html">Dashboard</a></li>
          <li><a href="events.html">Events</a></li>
          <li><a href="contact.html">Contact</a></li>
          <li><a href="feedback.html">Feedback</a></li>
        </ul>
      </nav>
    </div>
  </header>

  <main id="main-content" class="container">
    <section class="hero">
      <p class="muted">Your campus, organized</p>
      <h1>Welcome to StudentHub</h1>
      <p>Use the controls above to switch theme or open the menu. Your choices are
remembered.</p>
    </section>

    <!-- Slider -->
    <section aria-labelledby="slider-title">
      <h2 id="slider-title">Campus Highlights</h2>
      <div class="slider" id="slider" role="group" aria-roledescription="carousel" aria-
label="Campus highlights">
        <div class="slider-viewport">
          <div class="slider-track">
            <article class="slide slide-1" aria-label="1 of 3"><h3>Annual
TechFest</h3><p>Hackathons, demos, and guest talks.</p></article>
            <article class="slide slide-2" aria-label="2 of 3"><h3>Placement Drive</h3><p>Top
recruiters visit the campus in December.</p></article>
            <article class="slide slide-3" aria-label="3 of 3"><h3>Library Extended
Hours</h3><p>Open until 10 PM during examinations.</p></article>
          </div>
        </div>
        <div class="slider-controls">
          <button class="slider-btn" id="slide-prev" type="button" aria-label="Previous
slide">&#10094;</button>
          <div class="dots">
            <button class="dot" type="button" aria-label="Go to slide 1"></button>
            <button class="dot" type="button" aria-label="Go to slide 2"></button>
            <button class="dot" type="button" aria-label="Go to slide 3"></button>
          </div>
```

```
          <button class="slider-btn" id="slide-next" type="button" aria-label="Next
slide">&#10095;</button>
          <button class="slider-btn" id="slide-toggle" type="button" aria-label="Pause
slideshow">Pause</button>
        </div>
      </div>
    </section>

    <!-- Event cards that open the modal -->
    <section aria-labelledby="events-title">
      <h2 id="events-title">Upcoming Events</h2>
      <div class="card-grid">
        <article class="card">
          <h3>Web Dev Workshop</h3><p>12 Nov 2026</p>
          <button class="button open-modal" type="button" data-title="Web Dev Workshop" data-
date="12 Nov 2026, 10:00 AM, Lab 3" data-desc="A hands-on session on HTML, CSS, and JavaScript.
Bring your laptop.">View details</button>
        </article>
        <article class="card">
          <h3>Hackathon</h3><p>20 Nov 2026</p>
          <button class="button open-modal" type="button" data-title="24-Hour Hackathon" data-
date="20 Nov 2026, 9:00 AM, Main Auditorium" data-desc="Form teams of up to four and build a
solution to a campus problem.">View details</button>
        </article>
        <article class="card">
          <h3>Career Talk</h3><p>28 Nov 2026</p>
          <button class="button open-modal" type="button" data-title="Career Talk" data-
date="28 Nov 2026, 3:00 PM, Seminar Hall" data-desc="Alumni share interview tips and career
paths in software engineering.">View details</button>
        </article>
      </div>
    </section>

    <!-- FAQ -->
    <section aria-labelledby="faq-title">
      <h2 id="faq-title">Frequently Asked Questions</h2>
      <div class="faq" id="faq-list">
        <div class="faq-item">
          <h3><button class="faq-question" type="button" id="faq-q1" aria-expanded="false"
aria-controls="faq-a1">How do I register on StudentHub?</button></h3>
          <div class="faq-panel" id="faq-a1" role="region" aria-labelledby="faq-q1"><div
class="faq-inner"><p>Open the Register page, fill in your details, and submit the
form.</p></div></div>
        </div>
        <div class="faq-item">
          <h3><button class="faq-question" type="button" id="faq-q2" aria-expanded="false"
aria-controls="faq-a2">Where can I see my results?</button></h3>
          <div class="faq-panel" id="faq-a2" role="region" aria-labelledby="faq-q2"><div
class="faq-inner"><p>Results are listed on the Dashboard page after login.</p></div></div>
        </div>
        <div class="faq-item">
          <h3><button class="faq-question" type="button" id="faq-q3" aria-expanded="false"
aria-controls="faq-a3">How can I join a campus event?</button></h3>
          <div class="faq-panel" id="faq-a3" role="region" aria-labelledby="faq-q3"><div
class="faq-inner"><p>Open the Events page, choose an event, and follow its registration
link.</p></div></div>
        </div>
      </div>
    </section>
  </main>

  <!-- Modal -->
  <div class="modal-overlay" id="event-modal" aria-hidden="true">
    <div class="modal" role="dialog" aria-modal="true" aria-labelledby="modal-title" aria-
describedby="modal-desc">
      <h2 id="modal-title">Event title</h2>
      <p class="muted" id="modal-date"></p>
      <p id="modal-desc"></p>
      <button class="button" type="button" data-close>Close</button>
    </div>
```

```
  </div>

  <footer class="site-footer">
    <div class="container flex-row">
      <p>&copy; 2026 StudentHub | Student Portal UI Demonstration</p>
      <button class="button secondary" id="reset-prefs" type="button">Reset saved
preferences</button>
    </div>
  </footer>
</body>
</html>
```

3. Shared CSS (save as css/style.css)

```
/* Shared styles with theme variables (Practical 3 base + Practical 4 additions) */
:root {
 --primary: #174ea6;
 --link: #174ea6;
 --dark: #10233f;
 --page-bg: #f4f7fb;
 --surface: #ffffff;
 --text: #1e293b;
 --muted: #526174;
 --border: #d8e0ea;
}

/* Dark theme: only the variables change */
:root[data-theme="dark"] {
 --primary: #1f5fd0;
 --link: #8ab4f8;
 --dark: #0b1220;
 --page-bg: #0f172a;
 --surface: #1e293b;
 --text: #e2e8f0;
 --muted: #aab6c8;
 --border: #334155;
}

* { box-sizing: border-box; }
[hidden] { display: none !important; }

body {
 margin: 0;
 font-family: Arial, sans-serif;
 line-height: 1.6;
 color: var(--text);
 background: var(--page-bg);
 transition: background-color .3s ease, color .3s ease;
}
img { max-width: 100%; height: auto; }
a { color: var(--link); }
.container { width: min(1100px, 92%); margin-inline: auto; }
.skip-link { position: absolute; left: -9999px; top: 0; background: white; color: #000;
padding: .75rem; }
.skip-link:focus { left: .5rem; z-index: 30; }

/* Header, navigation, hamburger menu (mobile-first) */
.site-header { background: var(--primary); color: white; padding: 1rem 0; }
.header-row { display: flex; flex-wrap: wrap; align-items: center; justify-content: space-
between; gap: .75rem; }
.brand { margin: 0; font-size: 1.5rem; }
.header-controls { display: flex; gap: .5rem; }
.icon-btn { background: transparent; color: white; border: 1px solid white; border-radius:
.4rem; padding: .4rem .8rem; font: inherit; cursor: pointer; transition: background-color .2s
ease; }
.icon-btn:hover { background: rgba(255,255,255,.2); }
.main-nav { display: none; width: 100%; order: 3; }
.main-nav.open { display: block; }
```

```
.nav-list { display: flex; flex-direction: column; gap: .5rem; list-style: none; margin: 0;
padding: 0; }
.nav-list a { color: white; text-decoration: none; }
.nav-list a:hover, .nav-list a:focus { text-decoration: underline; }

main { padding: 1.5rem 0 2rem; }
section { margin-bottom: 2rem; }
.hero, .card, .panel { background: var(--surface); border: 1px solid var(--border); border-
radius: .75rem; padding: 1.25rem; }
h1, h2, h3 { line-height: 1.25; }
h1 { font-size: clamp(1.7rem, 4vw, 2.5rem); }
h2 { font-size: clamp(1.25rem, 3vw, 1.7rem); }
.muted { color: var(--muted); }
.card-grid { display: grid; grid-template-columns: 1fr; gap: 1rem; }
.flex-row { display: flex; flex-wrap: wrap; gap: .75rem; align-items: center; justify-content:
space-between; }

/* Buttons and cards with transition effects (intermediate extension) */
.button { display: inline-block; padding: .65rem 1rem; border-radius: .4rem; border: 1px solid
var(--primary); background: var(--primary); color: white; text-decoration: none; cursor:
pointer; font: inherit; transition: transform .2s ease, box-shadow .2s ease, background-color
.2s ease; }
.button:hover { transform: translateY(-2px); box-shadow: 0 4px 10px rgba(0,0,0,.25); }
.button.secondary { background: var(--surface); color: var(--link); border-color: var(--link);
}
.card { transition: transform .25s ease, box-shadow .25s ease; }
.card:hover { transform: translateY(-4px); box-shadow: 0 8px 18px rgba(0,0,0,.2); }

:focus-visible { outline: 3px solid #f5b700; outline-offset: 2px; }

/* Notification banner */
.notice-banner { display: flex; align-items: center; justify-content: space-between; gap: 1rem;
background: #fff3cd; color: #3d2f00; padding: .6rem 1rem; max-height: 6rem; overflow: hidden;
transition: opacity .3s ease, max-height .3s ease, padding .3s ease; }
.notice-banner p { margin: 0; }
.notice-banner.hide { opacity: 0; max-height: 0; padding-block: 0; }
.banner-close { background: none; border: 0; font-size: 1.5rem; line-height: 1; cursor:
pointer; color: inherit; padding: .25rem .5rem; }

/* Collapsible FAQ */
.faq-item { background: var(--surface); border: 1px solid var(--border); border-radius: .5rem;
margin-bottom: .75rem; }
.faq-item h3 { margin: 0; font-size: 1rem; }
.faq-question { width: 100%; text-align: left; background: none; border: 0; padding: 1rem
2.5rem 1rem 1rem; font: inherit; font-weight: bold; color: var(--text); cursor: pointer;
position: relative; }
.faq-question::after { content: "+"; position: absolute; right: 1rem; top: 50%; transform:
translateY(-50%); font-size: 1.4rem; transition: transform .3s ease; }
.faq-item.open .faq-question::after { transform: translateY(-50%) rotate(45deg); }
.faq-panel { display: grid; grid-template-rows: 0fr; visibility: hidden; transition: grid-
template-rows .3s ease, visibility .3s ease; }
.faq-item.open .faq-panel { grid-template-rows: 1fr; visibility: visible; }
.faq-inner { overflow: hidden; }
.faq-inner p { margin: 0; padding: 0 1rem 1rem; }

/* Slider */
.slider { background: var(--surface); border: 1px solid var(--border); border-radius: .75rem;
overflow: hidden; }
.slider-viewport { overflow: hidden; }
.slider-track { display: flex; transition: transform .5s ease; }
.slide { min-width: 100%; padding: 2.5rem 1.25rem; text-align: center; color: #fff; }
.slide h3 { margin-top: 0; font-size: 1.5rem; }
.slide-1 { background: linear-gradient(135deg, #174ea6, #10233f); }
.slide-2 { background: linear-gradient(135deg, #0f766e, #134e4a); }
.slide-3 { background: linear-gradient(135deg, #7c3aed, #4c1d95); }
.slider-controls { display: flex; flex-wrap: wrap; justify-content: center; align-items:
center; gap: .75rem; padding: .75rem; }
.slider-btn { background: var(--primary); color: #fff; border: 0; border-radius: .4rem;
padding: .5rem .9rem; font: inherit; cursor: pointer; }
```

```
.dots { display: flex; gap: .6rem; }
.dot { width: 1rem; height: 1rem; border-radius: 50%; border: 2px solid var(--link);
background: transparent; padding: 0; cursor: pointer; transition: background-color .2s ease; }
.dot.active { background: var(--link); }

/* Modal popup */
.modal-overlay { position: fixed; inset: 0; z-index: 20; display: grid; place-items: center;
padding: 1rem; background: rgba(0,0,0,.6); opacity: 0; visibility: hidden; transition: opacity
.25s ease, visibility .25s ease; }
.modal-overlay.open { opacity: 1; visibility: visible; }
.modal { width: min(480px, 100%); background: var(--surface); color: var(--text); border-
radius: .75rem; padding: 1.5rem; transform: translateY(20px); transition: transform .25s ease;
}
.modal-overlay.open .modal { transform: none; }
.no-scroll { overflow: hidden; }

.site-footer { background: var(--dark); color: white; padding: 1rem 0; margin-top: 1rem; }
.site-footer p { margin: 0; }

/* Tablet */
@media (min-width: 600px) {
 .card-grid { grid-template-columns: repeat(2, minmax(0, 1fr)); }
}

/* Desktop: menu always visible, hamburger hidden */
@media (min-width: 900px) {
 .card-grid { grid-template-columns: repeat(3, minmax(0, 1fr)); }
 .menu-toggle { display: none; }
 .main-nav { display: block; width: auto; order: 2; margin-left: auto; }
 .header-controls { order: 3; }
 .nav-list { flex-direction: row; gap: .5rem 1rem; }
}

/* Respect users who prefer reduced motion */
@media (prefers-reduced-motion: reduce) {
 * { transition: none !important; animation: none !important; }
}
```

4. JavaScript (save as js/script.js)

```
'use strict';

const $ = (selector) => document.querySelector(selector);
const PREF_KEY = 'studenthub-prefs';

/* ---------- Saved preferences (localStorage) ---------- */
function loadPrefs() {
  const defaults = { theme: null, bannerDismissed: false, openFaq: null };
  try {
    return { ...defaults, ...JSON.parse(localStorage.getItem(PREF_KEY) || '{}') };
  } catch (error) {
    return defaults; // storage blocked or data corrupted
  }
}

const prefs = loadPrefs();

function savePrefs() {
  try {
    localStorage.setItem(PREF_KEY, JSON.stringify(prefs));
  } catch (error) {
    /* ignore: the page still works without saving */
  }
}

/* ---------- 1. Light/Dark theme ---------- */
```

```
function initTheme() {
  const button = $('#theme-toggle');
  if (!button) return;

  const prefersDark = window.matchMedia('(prefers-color-scheme: dark)').matches;
  const apply = (theme) => {
    document.documentElement.setAttribute('data-theme', theme);
    button.setAttribute('aria-pressed', String(theme === 'dark'));
    button.textContent = theme === 'dark' ? 'Light' : 'Dark';
    button.setAttribute('aria-label', 'Dark theme');
  };

  apply(prefs.theme || (prefersDark ? 'dark' : 'light'));

  button.addEventListener('click', () => {
    const next = document.documentElement.getAttribute('data-theme') === 'dark' ? 'light' :
'dark';
    apply(next);
    prefs.theme = next;
    savePrefs();
  });
}

/* ---------- 2. Hamburger menu ---------- */
function initMenu() {
  const button = $('#menu-toggle');
  const nav = $('#main-nav');
  if (!button || !nav) return;

  const setOpen = (open) => {
    nav.classList.toggle('open', open);
    button.setAttribute('aria-expanded', String(open));
    button.innerHTML = open ? '&#10005;' : '&#9776;';
  };

  button.addEventListener('click', () => setOpen(!nav.classList.contains('open')));

  document.addEventListener('keydown', (event) => {
    if (event.key === 'Escape' && nav.classList.contains('open')) {
      setOpen(false);
      button.focus();
    }
  });

  // Reset the menu state when the screen becomes wide
  window.matchMedia('(min-width: 900px)').addEventListener('change', (event) => {
    if (event.matches) setOpen(false);
  });
}

/* ---------- 3. Notification banner ---------- */
function initBanner() {
  const banner = $('#notice-banner');
  const close = $('#banner-close');
  if (!banner || !close) return;

  if (prefs.bannerDismissed) {
    banner.hidden = true;
    return;
  }

  close.addEventListener('click', () => {
    banner.classList.add('hide');                      // CSS transition
    setTimeout(() => { banner.hidden = true; }, 300); // remove after animation
    prefs.bannerDismissed = true;
    savePrefs();
  });
```

```
}

/* ---------- 4. Collapsible FAQ (accordion) ---------- */
function initFaq() {
  const list = $('#faq-list');
  if (!list) return;

  const items = [...list.querySelectorAll('.faq-item')];
  const setItem = (item, open) => {
    item.classList.toggle('open', open);
    item.querySelector('.faq-question').setAttribute('aria-expanded', String(open));
  };

  if (prefs.openFaq !== null && items[prefs.openFaq]) setItem(items[prefs.openFaq], true);

  // Event delegation: one listener for all questions
  list.addEventListener('click', (event) => {
    const question = event.target.closest('.faq-question');
    if (!question) return;

    const item = question.closest('.faq-item');
    const willOpen = !item.classList.contains('open');
    items.forEach((other) => setItem(other, false));  // keep one answer open
    setItem(item, willOpen);

    prefs.openFaq = willOpen ? items.indexOf(item) : null;
    savePrefs();
  });
}

/* ---------- 5. Content slider ---------- */
function initSlider() {
  const slider = $('#slider');
  if (!slider) return;

  const track = slider.querySelector('.slider-track');
  const slides = [...track.children];
  const dots = [...slider.querySelectorAll('.dot')];
  const toggle = $('#slide-toggle');
  let index = 0;
  let timer = null;
  let playing = !window.matchMedia('(prefers-reduced-motion: reduce)').matches;

  const show = (n) => {
    index = (n + slides.length) % slides.length;
    track.style.transform = `translateX(-${index * 100}%)`;
    slides.forEach((slide, i) => slide.setAttribute('aria-hidden', String(i !== index)));
    dots.forEach((dot, i) => {
      dot.classList.toggle('active', i === index);
      dot.setAttribute('aria-current', String(i === index));
    });
  };

  const stopTimer = () => { clearInterval(timer); timer = null; };
  const startTimer = () => {
    stopTimer();
    if (playing) timer = setInterval(() => show(index + 1), 5000);
  };
  const updateToggle = () => {
    toggle.textContent = playing ? 'Pause' : 'Play';
    toggle.setAttribute('aria-label', playing ? 'Pause slideshow' : 'Play slideshow');
    track.setAttribute('aria-live', playing ? 'off' : 'polite');
  };

  $('#slide-prev').addEventListener('click', () => show(index - 1));
  $('#slide-next').addEventListener('click', () => show(index + 1));
  dots.forEach((dot, i) => dot.addEventListener('click', () => show(i)));
```

```
  toggle.addEventListener('click', () => { playing = !playing; updateToggle(); startTimer();
});

  slider.addEventListener('keydown', (event) => {
    if (event.key === 'ArrowLeft') show(index - 1);
    if (event.key === 'ArrowRight') show(index + 1);
  });

  // Pause while the user is hovering or focusing inside the slider
  slider.addEventListener('mouseenter', stopTimer);
  slider.addEventListener('mouseleave', startTimer);
  slider.addEventListener('focusin', stopTimer);
  slider.addEventListener('focusout', startTimer);

  show(0);
  updateToggle();
  startTimer();
}

/* ---------- 6. Modal popup ---------- */
function initModal() {
  const overlay = $('#event-modal');
  if (!overlay) return;

  const dialog = overlay.querySelector('.modal');
  let lastFocused = null;

  const focusable = () => [...dialog.querySelectorAll('button, [href], input, select,
textarea')];

  const openModal = (trigger) => {
    lastFocused = trigger;
    $('#modal-title').textContent = trigger.dataset.title;
    $('#modal-date').textContent = trigger.dataset.date;
    $('#modal-desc').textContent = trigger.dataset.desc;
    overlay.classList.add('open');
    overlay.setAttribute('aria-hidden', 'false');
    document.body.classList.add('no-scroll');
    focusable()[0].focus();
  };

  const closeModal = () => {
    overlay.classList.remove('open');
    overlay.setAttribute('aria-hidden', 'true');
    document.body.classList.remove('no-scroll');
    if (lastFocused) lastFocused.focus();   // return focus to the trigger
  };

  document.addEventListener('click', (event) => {
    const trigger = event.target.closest('.open-modal');
    if (trigger) openModal(trigger);
  });

  overlay.addEventListener('click', (event) => {
    if (event.target === overlay || event.target.closest('[data-close]')) closeModal();
  });

  document.addEventListener('keydown', (event) => {
    if (!overlay.classList.contains('open')) return;
    if (event.key === 'Escape') closeModal();
    if (event.key === 'Tab') {                // keep focus inside the dialog
      const items = focusable();
      const first = items[0];
      const last = items[items.length - 1];
      if (event.shiftKey && document.activeElement === first) { event.preventDefault();
last.focus(); }
```

```
      else if (!event.shiftKey && document.activeElement === last) { event.preventDefault();
first.focus(); }
    }
  });
}

/* ---------- Reset saved preferences (for testing) ---------- */
function initReset() {
  const button = $('#reset-prefs');
  if (!button) return;
  button.addEventListener('click', () => {
    try { localStorage.removeItem(PREF_KEY); } catch (error) { /* ignore */ }
    location.reload();
  });
}

document.addEventListener('DOMContentLoaded', () => {
  initTheme();
  initMenu();
  initBanner();
  initFaq();
  initSlider();
  initModal();
  initReset();
});
```

5. Component summary

Component	How it works (DOM / events)	Accessibility support
Hamburger menu	Click toggles the .open class on the nav and updates aria-expanded.	Button element, aria-controls, Escape key closes menu.
Theme switcher	Sets data-theme on <html>; CSS variables change. Saved in localStorage.	aria-pressed state; both themes keep sufficient contrast.
Notification banner	Close button adds .hide (transition), then sets hidden. Choice is saved.	role="status"; labelled close button.
Collapsible FAQ	One delegated click listener toggles .open and aria-expanded.	Heading + button pattern, aria-controls, hidden answers not read.
Content slider	translateX on the track; buttons, dots, arrow keys, optional autoplay.	Pause button, pause on hover/focus, respects reduced motion.
Modal popup	Content filled from data-* attributes; .open class shows overlay.	role="dialog", focus trap, Escape, overlay click, focus returns.

6. Extensions

Intermediate: Transition effects are added to cards and buttons using transform and box-shadow transitions (.card:hover and .button:hover), together with transitions on the FAQ panel, slider track, banner, and modal.

Advanced: User interface preferences are stored as one JSON object under the key studenthub-prefs (theme, bannerDismissed, openFaq). The object is read when the page loads and the saved theme, banner state, and open FAQ item are restored. The “Reset saved preferences” button in the footer clears the data for testing.

7. Browser console testing

Open Developer Tools (F12) and run these checks. The Console should show no errors.

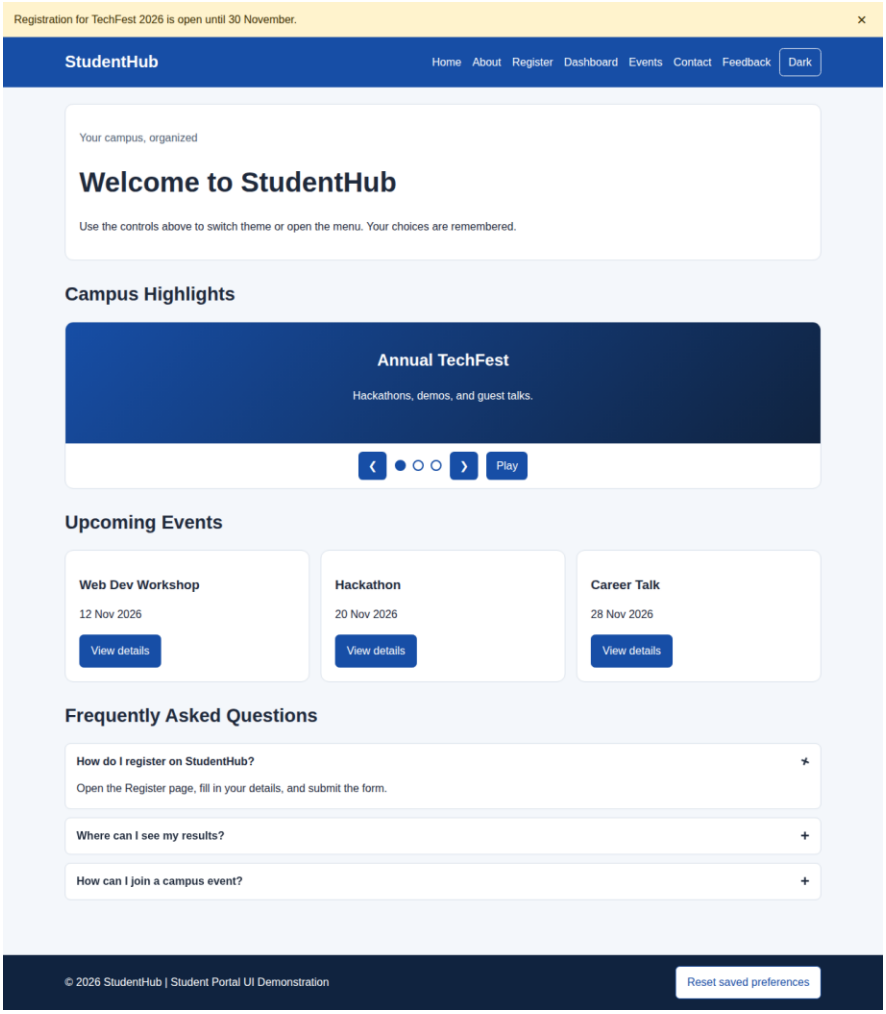
```
document.documentElement.dataset.theme          // "light" or "dark"
JSON.parse(localStorage.getItem('studenthub-prefs')) // saved preferences
document.querySelector('#menu-toggle').ariaExpanded // "true" / "false"
document.querySelectorAll('.faq-item.open').length  // 0 or 1
getEventListeners(document.querySelector('#faq-list')) // Chrome only
```

8. Implementation steps

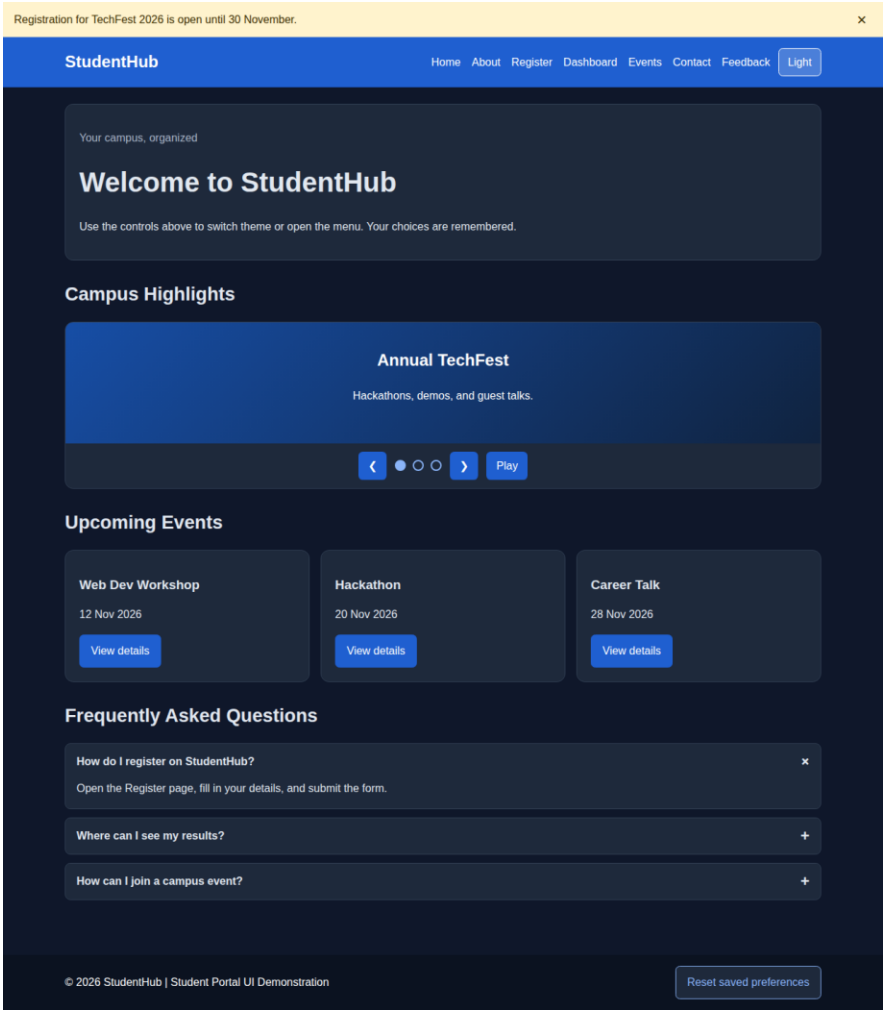
1. Open the StudentHub folder from Practical 3 in VS Code.
2. Create a js folder and the file js/script.js.
3. Add theme variables ([data-theme="dark"]) and the new component styles to css/style.css.
4. Add the banner, theme and menu buttons, slider, event cards, FAQ, and modal markup to index.html.
5. Link the script in the head with <script src="js/script.js" defer></script>.
6. Write one init function per component and call them all from a DOMContentLoaded listener.
7. Use classList.toggle() for open/closed states and keep ARIA attributes updated at the same time.
8. Save preferences with localStorage and restore them on page load.
9. Test with mouse, keyboard only (Tab, Enter, Space, Escape, arrows), and a narrow mobile width.
10. Check the Console for errors, then capture screenshots for the report and prepare the live demo.

C. Evidence of Work (Screenshots)

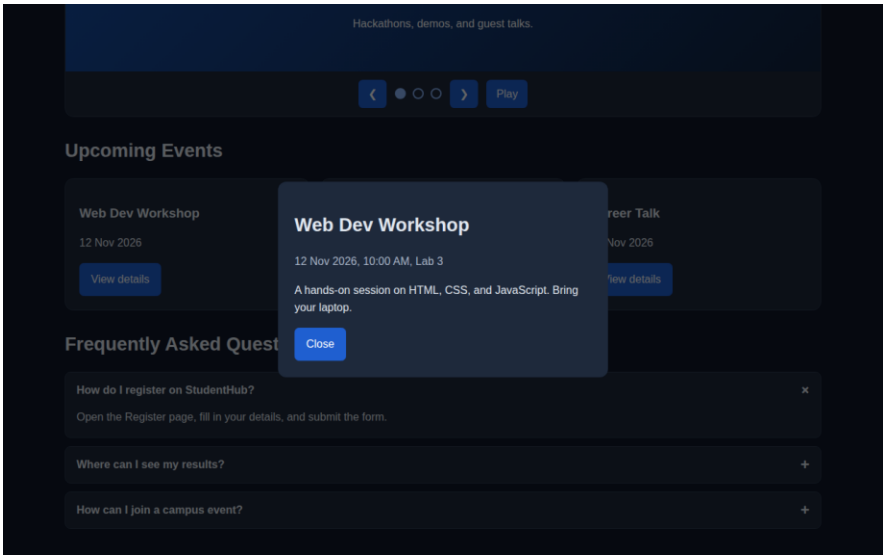
Screenshots of the page running in a browser. Replace or add your own screenshots from your machine, including VS Code and the Console.



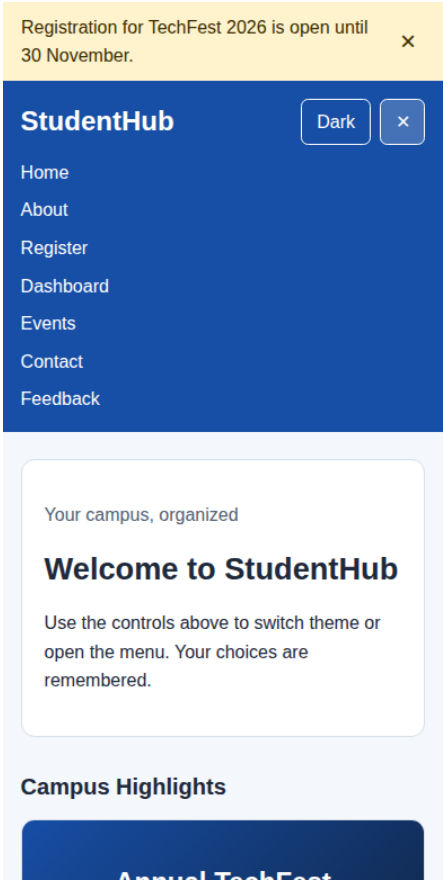
Home page — desktop view, light theme, first FAQ item open



Home page — desktop view, dark theme (restored after reload)



Modal popup opened from an event card



Hamburger menu open — mobile view

Also insert: VS Code project structure, and the Console / Application → Local Storage view showing studenthub-prefs.

D. Interactivity and Accessibility Checklist

☐ Script is loaded with defer and runs after the DOM is ready.

☐ Elements are selected correctly and no Console errors appear.

☐ Hamburger menu opens and closes and updates aria-expanded.

☐ Menu is always visible on desktop widths.

☐ Theme switcher changes the whole page using CSS variables.

☐ Theme and other preferences are saved and restored from localStorage.

☐ Notification banner can be dismissed and stays dismissed after reload.

☐ FAQ answers open and close with mouse and keyboard.

☐ Slider works with buttons, dots, and arrow keys, and can be paused.

☐ Modal traps focus, closes with Escape or overlay click, and returns focus.

☐ Transitions are present on cards and buttons and are disabled for reduced motion.

☐ Text and controls have sufficient contrast in both light and dark themes.

☐ Keyboard focus is clearly visible on every interactive control.

☐ No inline CSS or inline JavaScript event attributes (onclick) are used.

E. Challenges Faced

1. Keeping ARIA attributes in sync with the visual open/closed state.
2. Trapping keyboard focus inside the modal and returning it on close.
3. Animating FAQ height smoothly without a fixed max-height.
4. Avoiding a flash of the wrong theme when the page loads.
5. Making the slider usable (pause control, reduced-motion support).

F. Learning Outcomes

1. Learned to select and modify DOM elements using querySelector, classList, and attributes.
2. Applied addEventListener and event delegation correctly.
3. Used localStorage to remember the theme and other user preferences.
4. Built accessible interactive components with ARIA and keyboard support.
5. Added CSS transitions and tested behaviour using browser developer tools.

G. Viva Questions

What does querySelector return if nothing matches?

Answer: null.

What does classList.toggle() do?

Answer: It adds the class if it is missing and removes it if present.

Why use addEventListener instead of onclick attributes?

Answer: It keeps JavaScript out of HTML and allows several listeners on the same element.

What is event delegation?

Answer: Handling events for many child elements with one listener on their parent.

What is the purpose of event.preventDefault()?

Answer: It stops the browser’s default action, such as moving focus on Tab in the modal focus trap.

How do you store an object in localStorage?

Answer: Convert it with JSON.stringify() and read it back with JSON.parse().

What is the difference between localStorage and cookies?

Answer: localStorage is not sent to the server with every request and offers more space.

What does the defer attribute do?

Answer: It downloads the script while parsing HTML and runs it after parsing finishes.

Why are real button elements used for toggles?

Answer: They are focusable and respond to Enter and Space without extra code.

What does aria-pressed communicate?

Answer: The on/off state of a toggle button, such as the dark theme.

How does prefers-reduced-motion help users?

Answer: It lets the page remove animations for users who are sensitive to motion.

How can you debug an event listener that does not fire?

Answer: Check the selector in the Console, confirm the element exists, and inspect the Event Listeners panel.

H. Result

Interactive StudentHub components were implemented using JavaScript: hamburger menu, light/dark theme switcher, notification banner, collapsible FAQ, content slider, and modal popup. User preferences are saved in localStorage and restored on page load. Test the page with keyboard and mouse, check the Console for errors, and insert your own screenshots to demonstrate the final result.

Faculty Remarks

(Keep this section blank for faculty.)